Qn: 1. Try prompting using TL;DR and ELI-15 and observe the difference in output.

Answer 1a:

Context

Act like an experienced Software Architect and explain to me about Mercury Pub/Sub (Event Bus).

Instructions

-[MANDATORY] Refer to the attached documents in the project

-[STRICT] TL:DR Explain Mercury Pub\Sub with respect to this project.

Example:

Tone:

[STRICTLY] Professional tone in review comments and summary

Output:

Persona

Observation:

- Conciseness in the response and anchors the response to project context,
- Improves information Density
- Helps in Quicker Understanding
- Prioritizes important information and prevents generic explanations

Answer 2:

Context

Act like an experienced Software Archiect and explain to me about Mercury Pub/Sub (Event Bus).

Instructions

-[MANDATORY] ELI:15: Refer to the attached document and explain Mercury Pub\Sub with respect to this project.

Example:

Tone:

[STRICTLY] Professional tone in review comments and summary

Output:

Observation

Without ELI:15	**With ELI:15**
More technical	More understandable
Assumes expert knowledge	Assumes moderate knowledge
Dense architecture terminology	Simplified explanations
Heavy on implementation detail	Focus on conceptual clarity
Can feel abstract	Easier to visualize

Qn: 2. Using structured reasoning/step by step prompting to generate

a. Automation Framework Architecture

b. Test Strategy Document.

Answer 2a

2.a. Using chain of thoughts - a)Generate automation framework architecture

Prompt 1 :

Act as a Senior Test Automation Architect.

[MANDATORY] Design a scalable enterprise automation framework architecture using Playwright MCP for this project [[MANDATORY] Refer to all the project documents

[MANDATORY] Ask me 5 valid multiple choice questions one by one before generating

the final architecture.

[STRICTLY] ONCE AFTER ANSWERING THE FIRST QUESTION Go with next question[CRITICAL] Once after answering all questions generate automation framework

architecture

For each decision:

- explain reasoning

- explain tradeoffs

- explain best practices

[STRICT] Finally generate:

- high-level architecture

- folder structure

- technology recommendations

- implementation roadmap

- risks and mitigations

Great — before I design the enterprise-scale Playwright MCP automation framework architecture for the MySales 2.0 Quote-to-Hire platform, I need to clarify a few key architectural decisions that will significantly influence the framework design, scalability model, CI/CD strategy, observability, and maintainability.

I'll ask 5 multiple-choice questions one by one as requested.

Question 1 of 5

What is the primary automation execution strategy expected for this project?

A.

UI-heavy E2E automation focusing mainly on browser workflows

B.

Balanced UI + API + Event-driven automation with integrated validation across Mercury Pub/Sub, APIs, DB, and UI

C.

API-first automation with minimal UI validation

D.

Business-process orchestration testing with AI-assisted workflows, mocks, stubs, and synthetic environments

Please choose A, B, C, or D.

Excellent choice.
A balanced UI + API + Event-driven automation strategy is the strongest fit for this platform because the solution is heavily integration-centric with REST APIs, Mercury Pub/Sub events, Azure workflows, CQRS patterns, and multi-system orchestration. This aligns very closely with the project's architecture and non-functional requirements around resilience, integration reliability, observability, and workflow continuity.

Why this is strategically correct

This project includes:

- 60+ APIs and event integrations
- Mercury Pub/Sub event flows
- CRM outbound integrations
- TMS workflows
- BasePlan synchronization
- Azure Logic Apps orchestration
- CQRS/domain-event architecture
- Quote → Hire → Off-hire lifecycle orchestration

A UI-only framework would create:

- brittle tests
- slow regression cycles
- poor defect localization
- high maintenance overhead

A hybrid automation architecture enables:

- faster feedback loops
- shift-left validation
- service virtualization

- event validation
- resiliency testing
- better root-cause analysis

This is also aligned with the proposed Quality Engineering strategy in the solution documents.

Question 2 of 5

Which automation execution model should the framework prioritize?

A.

Centralized monolithic automation repository with shared libraries

B.

Domain-driven modular automation aligned to microservices/business domains (Quote, Hiring, Customer, Transport, Payments, etc.)

C.

Separate frameworks for UI, API, and Event testing

D.

Vendor-tool-centric framework using mostly low-code automation platforms

Please choose A, B, C, or D.

Interesting choice — and this introduces an important architectural tradeoff discussion.

Choosing separate frameworks for UI, API, and Event testing can provide strong specialization and independent scaling, especially in large enterprise ecosystems like MySales 2.0. However, it also increases governance complexity, duplication risks, and cross-framework orchestration overhead.

For this project specifically, the platform architecture is already highly distributed:

- Micro Frontends
- CQRS domain services
- Mercury Pub/Sub eventing
- Azure Logic Apps orchestration
- Multiple downstream integrations

- Distinct business domains like Quote, Hiring, Payments, Customer, Transport

So your choice suggests:

- specialized execution pipelines
- independent scaling
- optimized tooling per layer
- clear ownership boundaries

Key Tradeoffs

Benefit	Tradeoff
Faster execution specialization	Higher framework governance effort
Independent technology evolution	Shared utilities duplication risk
Better performance optimization	Harder end-to-end orchestration
Easier domain ownership	Reporting fragmentation
Independent CI/CD scaling	Cross-layer traceability challenges

Best-Practice Interpretation

For enterprise-scale systems like this, the strongest implementation pattern is usually:

Federated Automation Architecture

Meaning:

- Separate execution engines/pipelines
- BUT shared:
    - reporting layer
    - test data services
    - observability
    - reusable SDKs
    - auth handlers
    - environment management
    - Playwright MCP adapters
    - trace correlation

This avoids the biggest problem of fully isolated frameworks:

“automation silos.”

That consideration will strongly influence the final architecture I generate.

Question 3 of 5

What should be the primary test orchestration and execution approach?

A.

CI/CD-triggered execution only (Azure DevOps/GitHub Actions)

B.

Hybrid execution:

- CI/CD pipelines
- scheduled regression
- event-triggered execution
- environment-based orchestration

C.

On-demand manual execution by QA teams

D.

AI-agent-driven autonomous execution and self-healing pipelines

Please choose A, B, C, or D.

Excellent combination.
A hybrid orchestration model with both automated pipeline execution and controlled manual/on-demand execution is the most practical enterprise approach for this program.

This aligns especially well with the project’s:

- phased delivery model
- multiple SIT/UAT cycles
- complex downstream dependencies
- migration reconciliation gates
- integration-heavy workflows

- operational validation requirements

It also supports the proposed DevSecOps and QE operating model described in the project documents.

Why this is the right enterprise decision

CI/CD-driven execution enables

- Shift-left validation
- Fast feedback
- Pull-request quality gates
- Continuous regression
- Environment health validation
- Smoke and contract validation

Manual/on-demand execution enables

- UAT support
- Business walkthroughs
- Release certification
- Migration reconciliation validation
- Production shakedown testing
- Controlled execution during dependency instability

For a platform like MySales 2.0, fully autonomous execution would actually create operational instability because:

- external systems have dependency windows
- downstream systems may be unavailable
- Mercury events may require controlled sequencing
- transport/TMS integrations may not always support high-volume automation
- reconciliation checkpoints require human oversight

So this is a strong architectural direction.

Question 4 of 5

What should be the primary reporting and observability strategy for automation?

A.

Traditional HTML reports only (Playwright reports, Allure)

B.

Centralized enterprise observability:

- dashboards
- telemetry correlation
- event tracing
- API monitoring
- automation analytics
- defect intelligence

C.

Minimal reporting focused only on pass/fail

D.

AI-powered predictive quality analytics with self-healing insights

Please choose A, B, C, or D.

Strong enterprise-oriented choice.
This aligns extremely well with the platform's architecture, especially because the solution itself is designed around centralized observability and distributed tracing principles.

For this project, traditional test reporting alone would be insufficient because failures may originate from:

- UI layers
- BFF APIs
- Azure Logic Apps
- Mercury Pub/Sub events
- Redis cache
- downstream integrations
- CQRS handlers
- async workflows

- transport orchestration
- CRM synchronization
- BasePlan event propagation

A centralized observability-driven automation platform enables:

- root-cause analysis
- cross-system traceability
- event correlation
- environment diagnostics
- flaky test identification
- performance bottleneck tracking
- integration failure heatmaps
- release risk scoring

This becomes especially critical for:

- Quote → Hire → Off-hire orchestration
- event-driven processing
- eventual consistency validation
- distributed transaction visibility

The final framework architecture will therefore include:

- OpenTelemetry-style correlation concepts
- distributed execution telemetry
- centralized dashboards
- trace-linked evidence collection
- automation intelligence metrics
- environment observability hooks
- CI/CD analytics integration

This is fully aligned with the project's stated "Resilience & Observability" design principle.

Question 5 of 5

Which test environment strategy should the framework optimize for?

A.

Single shared QA environment

B.

Multi-environment enterprise strategy:

- Dev
- SIT
- UAT
- PERF
- DR
  with environment-aware execution

C.

Ephemeral containerized test environments per execution

D.

Production-like staging environments only

Please choose A, B, C, or D.

Answer 2B

Context

Act as a Senior QA Manager. Create a comprehensive Test Strategy Document for the Coates Project. Do refer to the attached document as an example. Refer to all the attached document in the project.

Instruction

[MANDATORY] Analyze the following areas sequentially as mentioned in the attached document.

[MANDATORY] Ask me 10 valid multiple choice questions one by one before generating the final Test Strategy

[STRICTLY] ONCE AFTER ANSWERING THE FIRST QUESTION Go with next question

[CRITICAL] Once after answering all questions, generate the document. Finally generate: - a professional Test Strategy Document as per the template referring to all the documents in the project.

Example

Tone:

[STRICTLY] Professional tone in review comments and summary

Output: should be as per the template

Persona

Act as a Proposal Manager to review the response and give the review comments

Answers

Excellent — I'll build a comprehensive enterprise-grade Test Strategy for the Coates MySales 2.0 Unified Quoting & Hiring Platform by referring to the BRD, SDD, RFx response documents, delivery methodology, integration architecture, AI strategy references, QE approach, DevSecOps model, Mercury Pub/Sub integration model, WCAG requirements, performance NFRs, DR expectations, and phased rollout strategy.

I will now ask 10 multiple-choice questions sequentially before generating the final professional Test Strategy document.

Question 1 of 10

What should be the PRIMARY test execution model for the Coates project?

A.

Centralized QE team executing all testing activities independently

B.

Hybrid Agile QE model with embedded QE engineers in Dev Pods and centralized governance

C.

Business-led testing with minimal automation focus

D.

Outsourced testing factory model with offshore-only execution

Great choice.
A Hybrid Agile QE model aligns strongly with the project's phased delivery model, Dev Pods structure, CI/CD approach, and domain-driven architecture defined for MySales 2.0. It also supports continuous testing, faster defect containment, and tighter integration validation across Mercury Pub/Sub, APIs, CRM, Baseplan, TMS, and Azure services.

Question 2 of 10

Which test automation strategy should be the PRIMARY enterprise approach for this program?

A.

UI-only automation using Selenium

B.

API-first automation with layered automation (API + UI + Event-driven validation)

C.

Manual regression with selective smoke automation

D.

Record-and-playback automation tools only

Excellent selection.
An API-first layered automation strategy is the best fit for the Coates ecosystem because the platform heavily depends on domain APIs, Mercury Pub/Sub events, Azure APIM integrations, asynchronous workflows, and distributed microservices. This also aligns with the proposed Qualtest-driven QE framework and DevSecOps delivery model.

Question 3 of 10

What should be the PRIMARY regression execution approach during release cycles?

A.

Full regression execution for every sprint

B.

Risk-based intelligent regression using impact analysis and critical business flows

C.

Only smoke and sanity testing before releases

D.

Manual regression execution during UAT only

Strong choice.
Risk-based intelligent regression is highly suitable for the phased Quote-to-Hire rollout because it optimizes execution effort while protecting critical business journeys such as Quote → Hire → Transport → Billing → Off-Hire. It also complements Qualtest-based impact analysis and CI/CD quality gates proposed for the program.

Question 4 of 10

Which performance testing strategy should be adopted for the platform?

A.

Basic load testing before production deployment only

B.

Continuous performance engineering with load, stress, spike, and endurance testing integrated into delivery phases

C.

Manual performance validation by business users

D.

Infrastructure-only performance monitoring without application testing

Excellent.
Continuous Performance Engineering is the right enterprise approach for Coates because the platform includes high-volume quoting flows, concurrent users, real-time availability APIs, Mercury Pub/Sub event processing, transport orchestration, and Azure PaaS auto-scaling requirements. This also aligns with the documented NFR expectations for response times, throughput, resilience, and scalability.

Question 5 of 10

What should be the PRIMARY non-functional testing priority for go-live readiness?

A.

Only functional stability

B.

Security, Performance, Accessibility, and Resilience validation together as release gates

C.

UI responsiveness only

D.

Database validation only

Perfect choice.
For Coates MySales 2.0, non-functional quality gates must collectively validate Security, Performance, Accessibility (WCAG 2.1 AA), Availability, DR readiness, and operational resilience because the platform handles customer data, pricing, payments, transport workflows, integrations, and enterprise-scale operations.

Question 6 of 10

Which environment strategy should be adopted for testing and release management?

A.

Single shared QA environment for all testing activities

B.

Dedicated multi-environment strategy (DEV, SIT, NFT, UAT/Pre-PROD, PROD, DR)

C.

Production-only validation model

D.

Ad-hoc temporary cloud environments without governance

Question 7 of 10 for the Test Strategy definition.

Considering the platform handles PII, payments, RBAC, external integrations, and PCI DSS scoped payment flows, which security testing strategy should be adopted?

A. Basic authentication testing only

B. Enterprise DevSecOps testing including SAST, DAST, RBAC validation, API security, secrets validation, dependency scanning, and penetration testing

C. Security testing only during UAT phase

D. Infrastructure vulnerability scanning without application-level validation

Question 8 of 10 for the Test Strategy definition.

What should be the preferred data migration testing and reconciliation strategy across the phased rollout model?

A. One-time bulk validation after final production migration

B. Phase-wise reconciliation with schema validation, ETL validation, audit checks, parallel-run validation, and business sign-offs at each migration gate

C. Manual spot checking of migrated records only

D. Migration validation limited to row-count comparison only

Question 9 of 10 for the Test Strategy definition.

Which test environment and release validation model should be followed for this multi-phase Azure-based enterprise implementation?

A. Single QA environment shared across all teams and phases

B. Dedicated environments with Dev, SIT, NFT, UAT, Pre-Prod, DR validation, production smoke validation, and release quality gates integrated into CI/CD

C. Production-only validation with feature toggles

D. UAT-only validation before deployment

Question 10 of 10 for the Test Strategy definition.

What should be the recommended governance and reporting approach for Quality Engineering across the Coates program?

A. Weekly manual status reporting through spreadsheets only

B. Centralized QE governance with real-time dashboards, defect analytics, traceability, release quality gates, risk-based reporting, and executive-level KPI visibility

C. Ad-hoc reporting managed independently by each scrum team

D. Governance limited only to defect counts and pass/fail metrics